

Introduction to Django Web Framework

1. What is Django and why is it called a high-level Python web framework?

Django is an open-source Python framework used to build web applications. It is called high-level because it handles many things automatically like database queries, authentication, sessions, and security. It follows the DRY (Don't Repeat Yourself) principle so we write less code. It comes with built-in features like ORM, admin panel, and templates. It is fast, secure, and helps in rapid development.

2. Explain the concept of MVT (Model-View-Template) architecture in Django.

Django follows MVT which stands for Model, View, Template. Model handles the database and tables. View contains the logic and controls what data should be shown. Template is the front-end part which displays the data in HTML. The URL dispatcher connects URLs to views. It separates data, logic, and presentation which makes code clean.

3. What are the main advantages of using Django over other frameworks?

Django allows rapid development because many things are ready-made. It is secure as it prevents CSRF, SQL injection, and XSS attacks. It has a powerful admin panel which makes managing data easy. Django is scalable, so big companies like Instagram use it. It has reusable apps and a large community for support.

4. What is the difference between Django settings.py and urls.py files?

settings.py contains all the configurations of the project like database setup, installed apps, middleware, static files. urls.py contains all the URL patterns and decides which view should handle which request. settings.py is for configuration, and urls.py is for navigation.

5. How does Django handle security features like CSRF protection and SQL injection prevention?

Django provides CSRF protection using CSRF tokens in forms. It prevents SQL injection by using ORM queries instead of raw SQL. It protects from XSS by auto-escaping HTML in templates. It provides password hashing and secure session handling. It also prevents clickjacking using security middleware.

Setting Up a Django Project and App

6. What is the difference between django-admin startproject and python manage.py startapp?

django-admin startproject is used to create a new project which contains settings.py, urls.py, wsgi.py, etc. python manage.py startapp is used to create a new app inside the project which contains models.py, views.py, templates, etc. Project is the container, app is a module like blog or shop.

7. Explain the structure of a Django project. What are the roles of settings.py, urls.py, and wsgi.py?

A project contains manage.py which is the command tool.settings.py has the configuration of the project.urls.py has the URL patterns and routes requests.wsgi.py is used when deploying the project on a server.Each app has its own models.py, views.py, and templates.

8. How do you configure INSTALLED_APPS in Django?

Go to settings.py file.Find the INSTALLED_APPS list.Add the app name as a string inside it, like 'blog'.This way Django knows about the app.

9. What is the purpose of manage.py file in a Django project?

manage.py is used to run commands for the project.It is used for running server, migrations, testing, etc.It loads the settings of the project automatically.It is like a shortcut for django-admin but project-specific.

10. If you create an app named blog, what steps are required to make Django recognize it?

First create app using python manage.py startapp blog.Add 'blog' in INSTALLED_APPS of settings.py.Create models, views, and templates inside blog.Add blog/urls.py and include it in project urls.py.Run makemigrations and migrate if models are created.

Building Models and URL Routing

11. How do you define a model in Django? Write a simple model for a Book with title, author, and published_date fields.

Models are written in models.py as Python classes.Each model represents a table in the database.

```
class Book(models.Model):  
    title = models.CharField(max_length=100)  
    author = models.CharField(max_length=100)  
    published_date = models.DateField()
```

12. What are Django migrations and why are they needed?

Migrations are a way to update database according to changes in models.They keep database schema and models.py in sync.Changes are stored in migration files.We don't need to write SQL manually.

13. How do you run migrations in Django?

- Run python manage.py makemigrations to create migration files.
- Run python manage.py migrate to apply changes to database.

14. What is the difference between `urls.py` at the project level and app level?

Project level `urls.py` is the main URL file which includes URLs of all apps. App level `urls.py` contains URLs related to that specific app only.

15. Write a URL pattern that maps the path `/books/` to a view named `book_list`.

```
urlpatterns = [  
    path('books/', views.book_list, name='book_list'),  
]
```